

CRYPTOLOGY ASSIGNMENT

Aditi Biju

261100690027

Level 1:

1. Write a Python program to accept five statements related to cryptology from the user and store them in a text file named cryptology.txt. Store each statement on a separate line.

sample statements:

Cryptology is the study of secure communication.

Cryptography focuses on protecting information.

Cryptanalysis focuses on analyzing cryptographic systems.

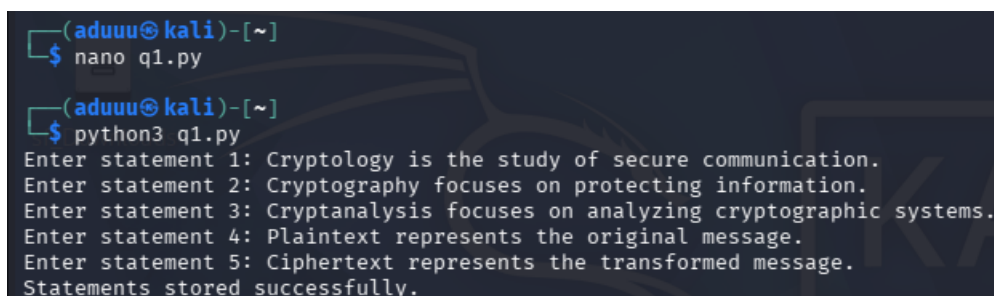
Plaintext represents the original message.

Ciphertext represents the transformed message.

Cryptanalysis is fun ;)

I love Cryptanalysis ;)

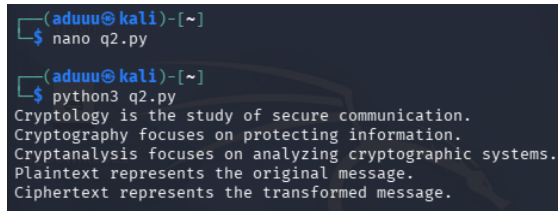
```
file = open("cryptology.txt", "w")  
  
for i in range(5):  
    statement = input("Enter statement {}: ".format(i + 1))  
    file.write(statement + "\n")  
  
file.close()  
  
print("Statements stored successfully.")
```



```
(aduuu@kali)-[~]  
$ nano q1.py  
  
(aduuu@kali)-[~]  
$ python3 q1.py  
Enter statement 1: Cryptology is the study of secure communication.  
Enter statement 2: Cryptography focuses on protecting information.  
Enter statement 3: Cryptanalysis focuses on analyzing cryptographic systems.  
Enter statement 4: Plaintext represents the original message.  
Enter statement 5: Ciphertext represents the transformed message.  
Statements stored successfully.
```

2. Write a Python program to read cryptology.txt and display its complete contents.

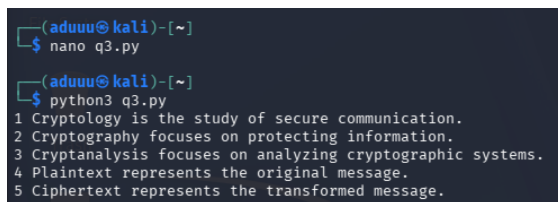
```
file = open("cryptology.txt", "r")  
  
content = file.read()  
  
print(content)  
  
file.close()
```



A terminal window on a Kali Linux system. The prompt is (aduuu@kali)-[~]. The user enters 'nano q2.py' to create a file. Then, they enter 'python3 q2.py' to run the program. The output of the program is displayed: 'Cryptology is the study of secure communication. Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems. Plaintext represents the original message. Ciphertext represents the transformed message.'

3. Write a program to read cryptology.txt one line at a time and display each line along with its line number.

```
file = open("cryptology.txt", "r")  
  
line_number = 1  
  
for line in file:  
  
    print(line_number, line.strip())  
  
    line_number += 1  
  
file.close()
```

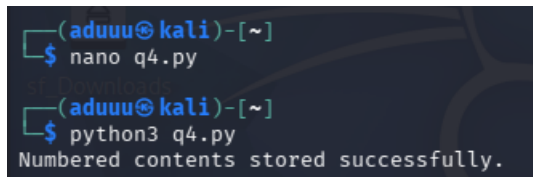


A terminal window on a Kali Linux system. The prompt is (aduuu@kali)-[~]. The user enters 'nano q3.py' to create a file. Then, they enter 'python3 q3.py' to run the program. The output of the program is displayed, showing line numbers and the content of each line: '1 Cryptology is the study of secure communication. 2 Cryptography focuses on protecting information. 3 Cryptanalysis focuses on analyzing cryptographic systems. 4 Plaintext represents the original message. 5 Ciphertext represents the transformed message.'

4. write a program to store the output in a file called cryptology_no.txt

```
file = open("cryptology.txt", "r")  
  
output = open("cryptology_no.txt", "w")  
  
line_number = 1  
  
for line in file:  
  
    output.write(str(line_number) + " " + line)  
  
    line_number += 1  
  
file.close()
```

```
output.close()
print("Numbered contents stored successfully.")
```

A terminal window with a dark background. The prompt is (aduuu@kali)~. The user enters 'nano q4.py' and then 'python3 q4.py'. The output is 'Numbered contents stored successfully.'

```
(aduuu@kali)~  
$ nano q4.py  
  
(aduuu@kali)~  
$ python3 q4.py  
Numbered contents stored successfully.
```

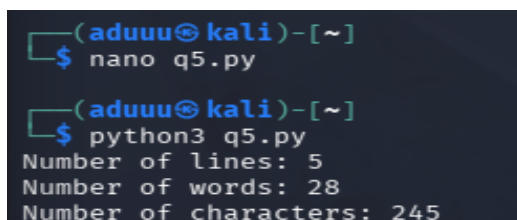
5. Write a program to read cryptology.txt and determine:

Number of lines

Number of words

Number of characters

```
file = open("cryptology.txt", "r")
lines = file.readlines()
number_of_lines = len(lines)
number_of_words = 0
number_of_characters = 0
for line in lines:
    number_of_words += len(line.split())
    number_of_characters += len(line)
print("Number of lines:", number_of_lines)
print("Number of words:", number_of_words)
print("Number of characters:", number_of_characters)
file.close()
```

A terminal window with a dark background. The prompt is (aduuu@kali)~. The user enters 'nano q5.py' and then 'python3 q5.py'. The output is 'Number of lines: 5', 'Number of words: 28', and 'Number of characters: 245'.

```
(aduuu@kali)~  
$ nano q5.py  
  
(aduuu@kali)~  
$ python3 q5.py  
Number of lines: 5  
Number of words: 28  
Number of characters: 245
```

6. Read cryptology_no.txt and count the number of:

Uppercase letters

Lowercase letters

Digits

Spaces

Special characters

```
file = open("cryptology_no.txt", "r")
```

```
uppercase = 0
```

```
lowercase = 0
```

```
digits = 0
```

```
spaces = 0
```

```
special = 0
```

```
for line in file:
```

```
    for ch in line:
```

```
        if ch.isupper():
```

```
            uppercase += 1
```

```
        elif ch.islower():
```

```
            lowercase += 1
```

```
        elif ch.isdigit():
```

```
            digits += 1
```

```
        elif ch.isspace():
```

```
            spaces += 1
```

```
        else:
```

```
            special += 1
```

```
print("Uppercase letters:", uppercase)
```

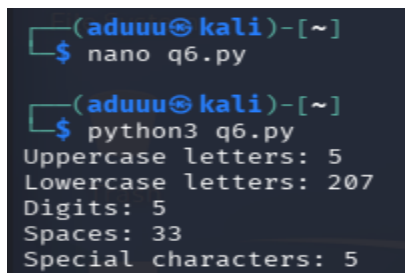
```
print("Lowercase letters:", lowercase)
```

```
print("Digits:", digits)
```

```
print("Spaces:", spaces)
```

```
print("Special characters:", special)
```

```
file.close()
```



```
(aduuu@kali)-[~]  
$ nano q6.py  
  
(aduuu@kali)-[~]  
$ python3 q6.py  
Uppercase letters: 5  
Lowercase letters: 207  
Digits: 5  
Spaces: 33  
Special characters: 5
```

7. Write a program that accepts a word from the user and searches for that word in cryptology.txt.

```
word = input("Enter the word to search: ")
```

```
file = open("cryptology.txt", "r")
```

```
found = False
```

```
for line in file:
```

```
    if word.lower() in line.lower():
```

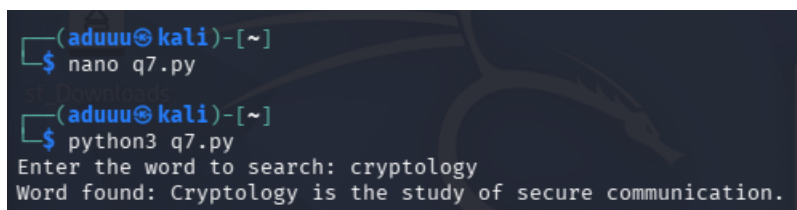
```
        print("Word found:", line.strip())
```

```
        found = True
```

```
if not found:
```

```
    print("Word not found.")
```

```
file.close()
```



```
(aduuu@kali)-[~]  
$ nano q7.py  
  
(aduuu@kali)-[~]  
$ python3 q7.py  
Enter the word to search: cryptology  
Word found: Cryptology is the study of secure communication.
```

8. Read cryptology.txt and display only the lines containing the word message.

```
file = open("cryptology.txt", "r")
```

```
for line in file:
```

```
    if "message" in line.lower():
```

```
        print(line.strip())
```

```
file.close()
```

```

(aduuiu@kali)-[~]
$ nano q8.py

(aduuiu@kali)-[~]
$ python3 q8.py
Plaintext represents the original message.
Ciphertext represents the transformed message.

```

9. Write a Python program to read cryptology.txt, convert all alphabetic characters to uppercase, and store the result in a new file called normalized.txt.

```

file = open("cryptology.txt", "r")
output = open("normalized.txt", "w")
for line in file:
    output.write(line.upper())
file.close()
output.close()
print("Contents converted to uppercase.")

```

```

(aduuiu@kali)-[~]
$ nano q9.py

(aduuiu@kali)-[~]
$ python3 q9.py
Contents converted to uppercase.

```

10. Write a program to read the file and print only the lines containing only alphabetic characters from the cryptology_no.txt file.

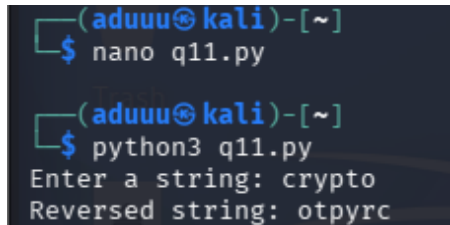
```

file = open("cryptology_no.txt", "r")
for line in file:
    line = line.strip()
    parts = line.split(" ", 1)
    if len(parts) == 2:
        text = parts[1]
        if text.replace(" ", "").isalpha():
            print(line)
file.close()

```

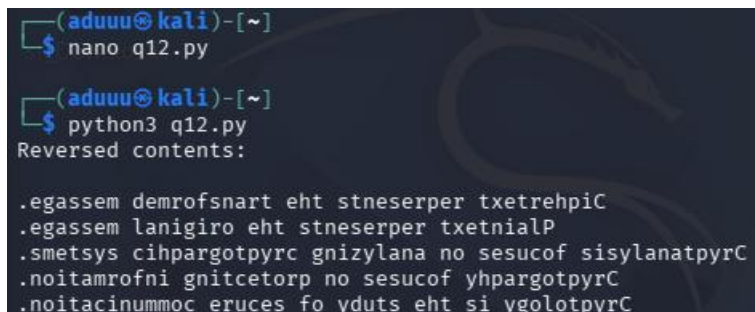
11. Write a program to get the string from the user and reverse the string.

```
text = input("Enter a string: ")
reverse = text[::-1]
print("Reversed string:", reverse)
```

A terminal window with a dark background and light blue text. The prompt is '(aduuu@kali)-[~]'. The user enters '\$ nano q11.py'. The prompt changes to '\$ python3 q11.py'. The user enters 'Enter a string: crypto'. The output is 'Reversed string: otpyrc'.

12. Improve the above code to reverse the string from the file.

```
file = open("cryptology.txt", "r")
text = file.read()
reverse = text[::-1]
print("Reversed contents:")
print(reverse)
file.close()
```

A terminal window with a dark background and light blue text. The prompt is '(aduuu@kali)-[~]'. The user enters '\$ nano q12.py'. The prompt changes to '\$ python3 q12.py'. The output is 'Reversed contents:' followed by five lines of reversed text: '.egassem demrofsnart eht stneserper txetrehpiC', '.egassem lanigiro eht stneserper txetnialP', '.smetsys cihpargotpyrc gnizylana no sesucof sisylanatpyrc', '.noitamrofni gnitcetorp no sesucof yhpargotpyrc', and '.noitacinummoc eruces fo yduts eht si ygotlpyrc'.

13. Simulate the tac command in Python.

```
file = open("cryptology.txt", "r")
lines = file.readlines()
for line in reversed(lines):
    print(line.strip())
file.close()
```

```

(aduuu@kali)-[~]
$ nano q13.py

(aduuu@kali)-[~]
$ python3 q13.py
Ciphertext represents the transformed message.
Plaintext represents the original message.
Cryptanalysis focuses on analyzing cryptographic systems.
Cryptography focuses on protecting information.
Cryptology is the study of secure communication.

```

Level 2: Regex

security_data.txt:

User alice logged in from 192.168.10.15

User bob logged in from 10.10.5.21

Contact admin@securitylab.com

Failed login attempts: 5

Session ID: SEC-2026-1045

Hash: a94f3c2d8e71b05f

Port 443 connection established

User admin failed authentication

1. Write a Python program to read security_data.txt and use a regular expression to identify and display all IPv4-address-like patterns present in the file.

```

import re

file = open("security_data.txt", "r")
text = file.read()

ip_addresses = re.findall(r'\b(?:\d{1,3}\.){3}\d{1,3}\b', text)

print("IP addresses:")

for ip in ip_addresses:
    print(ip)

file.close()

```

```

(aduuu@kali)-[~]
$ python3 regex1.py
IP addresses:
192.168.10.15
10.10.5.21

```

2. Write a Python program to identify and display all email-address-like patterns present in security_data.txt.

```
import re

file = open("security_data.txt", "r")

text = file.read()

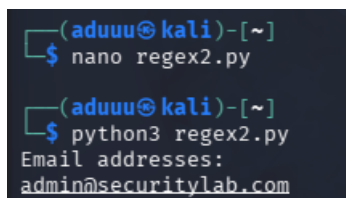
emails = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', text)

print("Email addresses:")

for email in emails:

    print(email)

file.close()
```

A terminal window with a dark background. The prompt is (aduuu@kali)~. The user enters 'nano regex2.py'. The prompt changes to (aduuu@kali)~/. The user enters 'python3 regex2.py'. The output is 'Email addresses:' followed by 'admin@securitylab.com' on the next line.

```
(aduuu@kali)~$ nano regex2.py
(aduuu@kali)~$ python3 regex2.py
Email addresses:
admin@securitylab.com
```

3. Write a Python program to extract session IDs that follow the format SEC-YYYY-NNNN, where YYYY and NNNN represent four digits.

```
import re

file = open("security_data.txt", "r")

text = file.read()

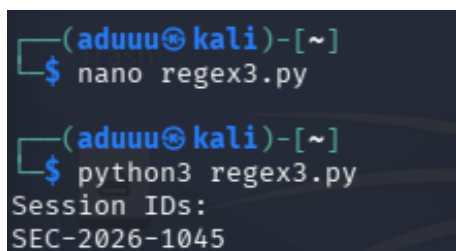
session_ids = re.findall(r'SEC-\d{4}-\d{4}', text)

print("Session IDs:")

for session in session_ids:

    print(session)

file.close()
```

A terminal window with a dark background. The prompt is (aduuu@kali)~. The user enters 'nano regex3.py'. The prompt changes to (aduuu@kali)~/. The user enters 'python3 regex3.py'. The output is 'Session IDs:' followed by 'SEC-2026-1045' on the next line.

```
(aduuu@kali)~$ nano regex3.py
(aduuu@kali)~$ python3 regex3.py
Session IDs:
SEC-2026-1045
```

4. Write a Python program to identify all hexadecimal sequences containing the characters 0–9, a–f, or A–F.

```
import re

file = open("security_data.txt", "r")

text = file.read()

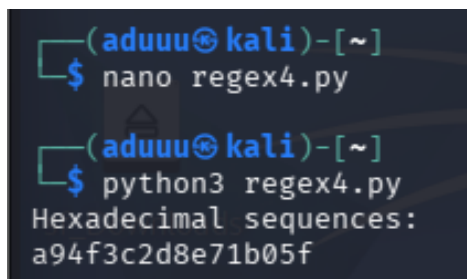
hex_values = re.findall(r'Hash:\s*([0-9a-fA-F]+)', text)

print("Hexadecimal sequences:")

for value in hex_values:

    print(value)

file.close()
```

A terminal window with a dark background. The prompt is (aduuu@kali)-[~]. The user enters \$ nano regex4.py. The prompt changes to (aduuu@kali)-[~]. The user enters \$ python3 regex4.py. The output is Hexadecimal sequences: a94f3c2d8e71b05f.

```
(aduuu@kali)-[~]
$ nano regex4.py

(aduuu@kali)-[~]
$ python3 regex4.py
Hexadecimal sequences:
a94f3c2d8e71b05f
```

5. Write a Python program to read security_data.txt and display all lines containing the word failed, irrespective of uppercase or lowercase.

```
import re

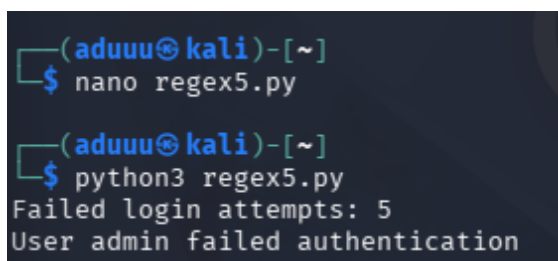
file = open("security_data.txt", "r")

for line in file:

    if re.search(r'failed', line, re.IGNORECASE):

        print(line.strip())

file.close()
```

A terminal window with a dark background. The prompt is (aduuu@kali)-[~]. The user enters \$ nano regex5.py. The prompt changes to (aduuu@kali)-[~]. The user enters \$ python3 regex5.py. The output is Failed login attempts: 5 and User admin failed authentication.

```
(aduuu@kali)-[~]
$ nano regex5.py

(aduuu@kali)-[~]
$ python3 regex5.py
Failed login attempts: 5
User admin failed authentication
```

6. Write a Python program to identify and extract the port number from statements of the form Port 443 connection established.

```
import re

file = open("security_data.txt", "r")

text = file.read()

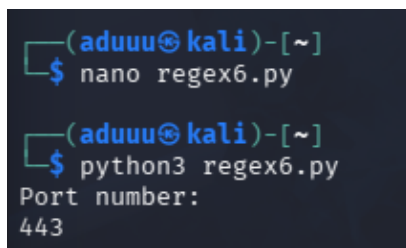
port = re.findall(r'Port\s+(\d+)\s+connection established', text)

print("Port number:")

for p in port:

    print(p)

file.close()
```

A terminal window with a dark background. The prompt is (aduuu@kali)~. The user enters 'nano regex6.py' and then 'python3 regex6.py'. The output of the script is 'Port number:' followed by '443' on the next line.

```
(aduuu@kali)~  
$ nano regex6.py  
(aduuu@kali)~  
$ python3 regex6.py  
Port number:  
443
```

Write a Python program to read the contents of security_data.txt and use regular expressions to extract IP-address-like patterns, email addresses, session IDs, and hexadecimal sequences. Store the identified patterns in a new file named pattern_results.txt.

```
import re

file = open("security_data.txt", "r")

text = file.read()

ip_addresses = re.findall(r'\b(?:\d{1,3}\.){3}\d{1,3}\b', text)

emails = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', text)

session_ids = re.findall(r'SEC-\d{4}-\d{4}', text)

hex_values = re.findall(r'Hash:\s*([0-9a-fA-F]+)', text)

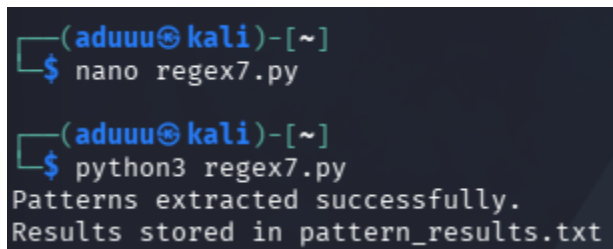
file.close()

output = open("pattern_results.txt", "w")

output.write("IP Addresses:\n")

for ip in ip_addresses:
```

```
    output.write(ip + "\n")
output.write("\nEmail Addresses:\n")
for email in emails:
    output.write(email + "\n")
output.write("\nSession IDs:\n")
for session in session_ids:
    output.write(session + "\n")
output.write("\nHexadecimal Sequences:\n")
for value in hex_values:
    output.write(value + "\n")
output.close()
print("Patterns extracted successfully.")
print("Results stored in pattern_results.txt")
```

A terminal window with a dark background and light blue text. The prompt is '(aduuu@kali)-[~]'. The first command is '\$ nano regex7.py'. The second command is '\$ python3 regex7.py'. The output of the script is displayed: 'Patterns extracted successfully.' and 'Results stored in pattern_results.txt'.

```
(aduuu@kali)-[~]
$ nano regex7.py

(aduuu@kali)-[~]
$ python3 regex7.py
Patterns extracted successfully.
Results stored in pattern_results.txt
```